

GPT-6 Orchestration over ROS 2

Task Planning, Code Repair, and Single-Step Arm Correction

Research Proposal III · Zimo Wen

September 2026

Abstract

This proposal studies a robot system in which GPT-6 coordinates the complete task, generates and revises executable control code, and uses execution feedback to decide when the robot needs a local correction. ROS 2 connects observation, action execution, and the existing low-level controllers. The robot can continue a valid local policy without making a language-model request on every control tick. When a task assumption fails, GPT-6 receives a bounded observation trace, corrects the relevant code or action reference, and submits a versioned replacement for execution. The research question is whether selective, feedback-grounded repair improves task completion over one-shot generated ROS programs and full replanning, at a matched sensing, action, and inference budget. This is a proposed architecture and experimental protocol; no ROS integration or performance result is claimed here.

1. Scope and Relationship to the Three Proposals

The requested division of work. Proposal I studies bottom-level manipulation: direct local actions, grounded force–position requirements, and learned contact policies. Proposal II studies immediate response: agent-generated state machines, stateful feedback programs, and locally solved constraints. Proposal III studies **GPT-6 task orchestration and code repair connected to a ROS execution system**. In-context learning is not a standalone proposal in this collection.

GPT-6 stays in the deployed system. GPT-6 plans task stages, produces code against a declared ROS-facing API, interprets allowed execution feedback, and makes the final model-authored code correction. The revised program then passes deterministic interface checks before ROS executes it. These checks do not replace GPT-6’s reasoning, and GPT-6’s approval does not establish physical success. The exact GPT-6 deployment, prompts, reasoning settings, and budgets are frozen and recorded for each evaluation.

ROS has an explicit role. ROS 2 transports timestamped observations and action feedback, manages the execution lifecycle, and connects to robot controllers through a declared driver. A controller such as a servo, trajectory controller, or supported force controller realizes the commands. ROS itself is not a learned policy and does not make a task plan automatically. The generated program must state what it observes, which interface it commands, and what feedback changes its behavior.

Meaning of a single correction. The default study uses a bounded correction to the next end-effector reference or a short declared action segment. Translation, orientation, frame, duration, and gripper behavior are explicit. A joint-space interface can instead support a bounded joint increment, but the two command spaces are separately specified and evaluated. “Single-step” does not mean that a remote model participates in every servo cycle.

Nominal behavior. A nominal action may come from an existing ROS skill, a generated program, or an optional frozen VLA. A VLA is not required by this proposal. The earlier GPT-reviewed-VLA draft is retained as a narrower historical variant; this revision makes whole-task orchestration, code repair, and the ROS boundary explicit.

2. Architecture and Observable Execution Contracts

Observation gateway. A ROS-facing adapter assembles a coherent observation from approved camera streams, measured joint state, gripper state, and available force sensing. Each observation carries acquisition time, frame identifiers, and calibration version. Estimated object geometry is labeled as an estimate. GPT-6 receives only the permitted subset; simulator state, success predicates, hidden object poses, and evaluator-only event schedules are excluded, including debug topics that expose them.

Task supervisor. GPT-6 converts the instruction and current observation into task stages with observable prerequisites, termination conditions, and escalation conditions. The supervisor maintains task identity, current stage, policy version, and the most recent accepted observation. A failed grasp can require local adjustment; a changed user instruction can require a new stage plan. The system logs these as different interventions rather than calling every intervention a correction.

Generated code. The initial executable is a ROS-facing task node or a restricted feedback-policy module hosted by a fixed ROS node. It uses a narrow API for observation, short motion references, gripper commands, cancellation, and feedback. It does not modify the driver or the real-time controller implementation during a live control cycle. GPT-6 may patch a local policy function or stage transition, but must name the old version, the evidence for the edit, and the intended behavioral difference.

ROS execution layer. A practical implementation can use ROS 2 Actions for goal, feedback, result, and cancellation; `tf2` for calibrated frame transforms; and `ros2_control` or a vendor-supported controller for command execution. MoveIt Servo is a candidate for compatible local motion interfaces. Each component must be validated for the chosen arm. Action cancellation being accepted does not imply that physical motion has already stopped; the executor waits for the declared controller acknowledgement and measured state before handing over ownership.

State-valid repair. A repair record includes

$$r = (v_{\text{base}}, t_{\text{obs}}, \mathcal{F}, \Delta u, H, \mathcal{P}, \mathcal{T}),$$

where v_{base} is the program/action version being amended, t_{obs} is the observation timestamp, $\mathcal{F}$ is the coordinate frame, Δu is the bounded edit, H is its validity horizon, $\mathcal{P}$ contains observable preconditions, and $\mathcal{T}$ describes termination and handoff. A result based on an obsolete version cannot overwrite a newer action. If freshness or preconditions fail, execution requests a new observation instead of applying the stale edit.

Control ownership. Exactly one executor owns each arm’s active motion reference. The local controller continues at its measured supported rate; GPT-6 requests occur asynchronously at stage boundaries or escalation events. During a pending model call, a declared locally valid policy or hold behavior remains responsible for the arm. Threading alone is not evidence of responsiveness: timestamps must show when a changed observation actually changes a physical command.

3. Three Repair Mechanisms within the Same System

A: parameter and single-step correction. GPT-6 preserves the program and adjusts a declared next-step reference, such as an approach offset, an orientation increment, or a calibrated controller parameter. For an EEf edit, composition follows a specified tool or world frame rather than adding untyped pose arrays. Joint increments form a separately labeled variant. This establishes whether a small explicit correction is sufficient before crediting unrestricted program synthesis.

B: bounded code patch. GPT-6 changes a policy function or a stage transition while retaining unaffected code. For example, replace a fixed approach target with a fresh observed target and a bounded alignment loop. A patch declares the input observations, output command space, applicable stage, and maximum execution duration. Syntax, supported calls, frame/unit consistency, and resource bounds are checked before installation. Invalid patches consume the declared repair budget and remain in the logs.

C: supervisor-assisted stage repair. If the local policy’s assumptions no longer hold, GPT-6 revises the stage plan and generates its replacement ROS-facing program. It then performs a final code review using the same fixed model configuration and the current permitted evidence. The second call is not independent verification: an ablation must establish whether review reduces harmful edits enough to justify its cost. This broader repair is reported separately from a single-step correction.

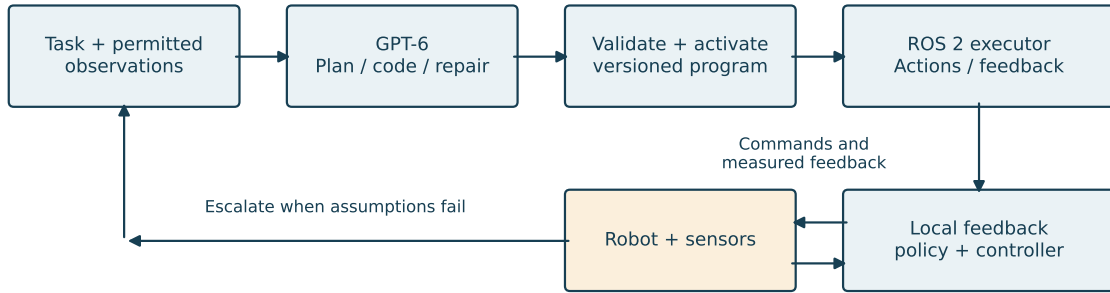


Figure 1: GPT-6 provides whole-task orchestration and code repair. ROS executes a versioned program and routes current feedback to the local policy and supervisor. Fast control remains inside the ROS/controller loop. The diagram describes the proposed mechanism, not an implemented benchmark result.

Common lifecycle. Plan, generate, validate, execute, observe, and either continue or repair. Final model-authored correction precedes interface validation and controlled activation. The previous program and rejected candidates remain available for diagnosis. A code rollback is permitted only when its assumptions still match the current physical state; restoring old source code does not restore the world.

4. Experimental Design and Matched Comparisons

Implementation sequence. First build an observation/action adapter and validate its semantics independently of GPT-6: frames, command scaling, gripper state, clock handling, and cancellation acknowledgement. Next connect generated task programs and single-step corrections. Finally enable bounded code patching and stage repair. Existing LIBERO and RoboTwin environments can support separate ROS-adapter research runs; their current non-ROS runs are not evidence for this architecture. Real-arm evaluation requires a validated driver and task-specific operating envelope.

Task families. Include grasp–place with changed object position, placement alignment, interrupted transport, and compatible articulated-object operations. Add contact tasks only where the sensing and controller semantics have been established. Dynamic conditions include target displacement, brief observation loss, and an instruction change. The evaluator controls disturbance schedules privately and verifies that tasks remain physically meaningful.

Proposed diagnostic panel. Start with 12 task families, four conditions per family (one unperturbed and three declared disturbances), and five seeds: 240 episodes per method. This is an experimental design, not a completed dataset. Development tasks determine disturbance magnitudes and repair limits. Hold out task–condition combinations for evaluation, group related recipes together, and report whether a task’s nominal skill was already in the library. Use the same starting states and disturbance schedules across methods where possible.

Comparison	Question isolated
Fixed ROS task policy	What does the existing executable system achieve without model-generated repair?
GPT-6 one-shot ROS program	Does initial program generation suffice?
GPT-6 full replanning	Does local repair improve on regenerating the program after every escalation?
GPT-6 parameter repair only	Are small action corrections sufficient?
GPT-6 bounded code repair	Does changing feedback logic add value beyond parameter edits?
Full supervisor with final review	Does orchestration plus reviewed repair justify its extra inference?

Fairness. Keep the ROS driver, sensing, motion primitives, controller limits, model snapshot, and nominal skill library fixed. Match action and model-call budgets in the primary comparison; separately report actual incurred costs. If an optional VLA is used, freeze it across the compared methods and identify that experiment as a VLA-backed variant. A direct-SDK adapter comparison can estimate ROS transport overhead under the same control semantics; it does not by itself evaluate GPT-6’s reasoning.

Standard task reporting. Keep the custom ROS/disturbance panel separate from standard LIBERO and RoboTwin results. Any standard LIBERO short leaderboard uses the declared 120-task pass@10 protocol with subset and suite breakdowns. Do not merge the custom panel, single-step diagnostics, and standard benchmark episodes into one favorable success figure.

5. Measurements, Ablations, and Result Interpretation

Primary outcomes. Report final simulator/harness task success, completion time, and useful versus harmful interventions. A compiled program, an accepted ROS Action, or GPT-6’s approval is not task success. Infrastructure, transport, missing-package, and interrupted attempts are separately categorized and excluded from task denominators. Preserve failed programs, ROS observations, action traces, and exact launch/model manifests.

Responsiveness. Measure event-to-observation, observation-to-detection, detection-to-model-request, model turnaround, patch validation, cancellation acknowledgement, and first effective corrected command. Report median and tail latency and the number of missed validity windows. An experiment that pauses simulation while GPT-6 reasons is labeled separately; it cannot establish responsiveness in a changing physical scene.

Repair quality. Measure patch acceptance, execution errors, local recovery, subsequent task success, intervention magnitude, and harm to actions that would otherwise succeed. In a simulator-only diagnostic, evaluate the original and corrected continuation from an evaluator-owned checkpoint. Lock the correction decision before the hidden paired outcomes are computed. The branch outcomes remain post-hoc labels and are never returned to GPT-6 for online selection.

Efficiency. Record prompt, completion, and total tokens; model and perception calls; unmetered calls; wall time; and perception, action, recovery, and review time. Measure the cost of a generated program that keeps working without another model call. A smaller token count caused by premature failure is not an efficiency improvement. Require success and intervention harm to remain acceptable when comparing cheaper variants.

Ablation A: what does GPT-6 contribute? Compare one-shot generation, full regeneration, bounded patches, and parameter edits with identical primitives. Remove the final GPT-6 code-review call while matching total inference budget in one experiment. If a fixed local monitor with parameter updates performs equally well, the evidence supports that simpler mechanism rather than a broad code-repair claim.

Ablation B: is ROS responsible for the gain? Hold the observation and command interface fixed while changing the adapter, where a direct SDK implementation is available. Separate middleware latency, action-management correctness, and model decision quality. Replacing an incomplete blocking driver with a better driver is an engineering improvement, but must not be attributed to GPT-6-generated reasoning.

Ablation C: state freshness and correction granularity. Compare version-checked activation with a separately labeled stale-edit diagnostic, single-step edits with short-segment edits, and parameter-only with code-level changes. Test latency and observation noise at development-calibrated levels. Report both the range in which local repair helps and the conditions in which the task requires broader replanning.

Possible outcomes. Improved completion with fewer full replans supports selective repair. Equal performance from one-shot programs weakens the repair hypothesis. A review call that increases latency without reducing harmful edits is not justified. Gains only under paused simulation identify a latency limitation. Better local alignment without better final success is insufficient evidence for the proposed system.

6. Contribution, Boundaries, and Reproducibility

Intended contribution. The research contribution is a measured way to connect GPT-6 task reasoning and code repair to state-valid robot execution: explicit repair scope, observable applicability, program/action version ownership, and controlled ROS handoff. ROS integration alone, ordinary feedback loops, and generic code generation are established ideas. Novelty must be demonstrated against matched implementations of those components rather than asserted from their combination.

Distinct scope of the three papers. The first paper changes what the local policy commands and how contact requirements are represented. The second changes how generated local behavior responds to new observations without waiting for another model call. This third paper changes how GPT-6 coordinates those behaviors, decides that they need repair, and installs corrected code through a ROS system. Shared infrastructure may support all three; each paper needs an experiment that isolates its own variable.

Model configuration. Record the actual GPT-6 deployment identifier, supported input modalities, prompts, sampling and reasoning settings, maximum repair rounds, and tool budget. Structured perception is sufficient for a text interface; raw images are supplied only to a deployment that supports them. The name GPT-6 is not a latency, accuracy, or control-rate guarantee. Model services and local controllers have different timing responsibilities.

Physical and statistical limits. Single-step EEF or joint corrections do not establish torque-control ability. Force references require calibrated units, frames, sensing, and a controller with the relevant semantics. Simulation gains do not establish real-arm contact performance. Report paired task-level intervals and cluster related task variants; multiple frames or checkpoints from one episode are not independent successes. Test-set patches are confined to the current episode and do not update the frozen skill library.

Artifacts to retain. Release the ROS message/action contracts, adapter versions, approved observation allowlist, generated programs and patches, model configuration, controller configuration, and evaluation manifest. Separate development corrections from frozen evaluation. Keep rejected and interrupted runs visible. Hidden ground truth remains with an independent evaluator and outside GPT-6, generated programs, and reviewer-visible context.

Status of this document. This revision replaces the third proposal’s VLA-specific emphasis with the requested GPT-6–ROS orchestration and code-repair system, while retaining VLA-backed actions as an optional variant. The first two proposals remain bottom-level manipulation and reactive-policy synthesis. This document does not claim that ROS has been integrated into the currently running remote experiments, and it introduces no measured success rate.

References

- [1] Code as Policies: Language Model Programs for Embodied Control. <https://arxiv.org/abs/2209.07753>
- [2] ROS 2 documentation: Actions, Quality of Service, and tf2. <https://docs.ros.org/en/jazzy/>
- [3] ros2_control: controller and hardware interface documentation. <https://control.ros.org/jazzy/>
- [4] MoveIt Servo: real-time motion-command interface documentation. https://moveit.picknik.ai/main/doc/examples/realtime_servo/realtime_servo_tutorial.html
- [5] LIBERO: Benchmarking Knowledge Transfer for Lifelong Robot Learning. <https://arxiv.org/abs/2306.03310>